השוואת מחירים: איך זה עובד

הסבר מפורט על האפליקציה ועל שרת המחירים, עם תרשימים · נובמבר 2026

המערכת בנויה משני חלקים. **האפליקציה** רצה בטלפון: היא מורידה את מחירי הסניפים שבחרתם, שומרת אותם במאגר מקומי, ומשווה ביניהם. **השרת** הוא רכיב אופציונלי שרץ על מחשב: הוא מוריד מראש את הקבצים של הרשתות ומגיש אותם לאפליקציה מהר, ובנוסף מציע חיפוש והשוואה דרך ממשק API ודף אינטרנט. שני החלקים קוראים את אותם קבצים, אותם הרשתות חייבות לפרסם לפי חוק המזון (שקיפות מחירים, 2014).

1. מקור הנתונים: קובצי השקיפות של הרשתות

כל רשת עם 3 סניפים ומעלה מפרסמת כל יום קובצי XML דחוסים מכמה סוגים. המערכת קוראת כרגע 19 רשתות, כ-1,585 סניפים:

סוג קובץ	מה יש בו	בשימוש
PriceFull	כל המוצרים בסניף: ברקוד, שם, יצרן, מחיר, מחיר ליחידת מידה, תאריך עדכון המחיר	כן, פעם ביום לכל סניף
PromoFull	כל המבצעים הפעילים בסניף: תיאור, תאריכים, כמות מינימלית, מחיר מבצע, מועדון, ורשימת הברקודים שבמבצע	כן, פעם ביום לכל סניף
Stores	רשימת הסניפים של הרשת: מספר, שם, כתובת, וסמל יישוב (קוד של הלמ"ס, למשל 5000 = תל אביב)	כן, לבחירת סניפים
Price / Promo	עדכונים חלקיים במהלך היום	לא; מספיק הקובץ המלא היומי

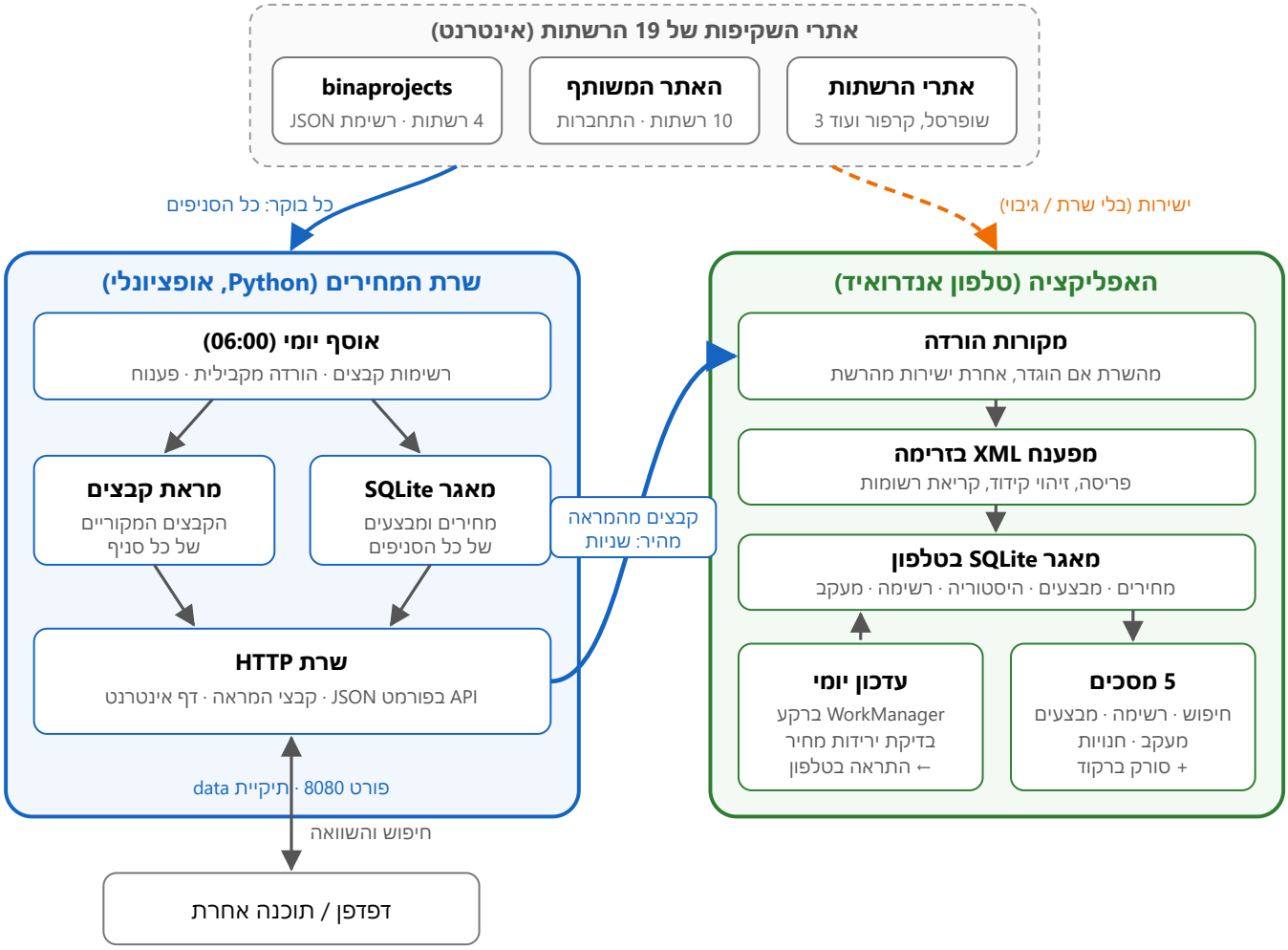
שם הקובץ מקודד את התוכן, למשל `PriceFull7290027600007-001-001-20261106-043100`: סוג הקובץ, מספר הרשת (שופרסל), תת-רשת, מספר הסניף, ותאריך ושעת הפרסום. לפי השם המערכת יודעת לאיזה סניף שייך כל קובץ ואם הוא חדש מזה שכבר נטען. יש רשתות שחורגות מהתבנית: קשת טעמים משמיטה את התת-רשת, וחצי חינם קוראת לרשימת הסניפים StoresFull. המערכת מטפלת בשתיהן.

איפה כל רשת מפרסמת, ואיך המערכת מוצאת את הקבצים

אתר	רשתות (מספר סניפים)	איך זה עובד
האתר המשותף `url.publishedprices.co.il`	רמי לוי (99), דור אלון (157), פרש מרקט (73), טיב טעם (54), יוחננוף (51), קשת טעמים (28), אושר עד (24), סאלח דבאח (12), סטופ מרקט (11), פוליצר (8)	מתחברים עם שם המשתמש הציבורי של הרשת (למשל `RamiLevi`, בלי סיסמה), מקבלים קוד אבטחה (csrf), ומבקשים את רשימת כל הקבצים בפורמט JSON. ההורדה נעשית באותו חיבור; אם החיבור פג, המערכת מתחברת מחדש.
binaprojects	קינג סטור (48), מעיין אלפיים (38), זול ובגדול (35), שוק העיר (26)	רשימת JSON של 1,000 הקבצים האחרונים. לכל קובץ יש עמוד שמחזיר את הכתובת האמיתית שלו, וממנה מורידים.
שופרסל	שופרסל (417)	טבלת קבצים מסוננת לפי סוג וסניף, 20 לעמוד, עם קישורים חתומים לאחסון בענן. איטי מאוד: כ-50 שניות לעמוד ולפעמים שגיאה 502, ולכן יש ניסיונות חוזרים וזמני המתנה ארוכים.
סופר-פארם	סופר-פארם (307)	טבלה שמסננים לפי סוג קובץ ותאריך, 20 שורות לעמוד; המערכת עוברת על כל העמודים של היום.

קרפור (לשעבר יינות ביתן)	קרפור (147)	עמוד לכל יום עם כל הקבצים של אותו יום. מוקדם בבוקר, אם הקובץ של היום עוד לא פורסם, נלקח הקובץ של אתמול.
וולט	וולט מרקט (37)	גם כאן עמוד לכל יום, עם קישורי הורדה ישירים.
חצי חינם	חצי חינם (13)	עמודים של 50 קבצים לכל סוג (מחירים, מבצעים, סניפים), מהחדש לישן.

2. התמונה הכוללת



תרשים 1: רכיבי המערכת וזרימת הנתונים. החץ הכחול הוא המסלול המהיר דרך השרת; החץ הכתום המקווקו הוא הורדה ישירה מהרשת.

בלי שרת, הטלפון מוריד כל קובץ ישירות מאתר הרשת. זה עובד, אבל בשופרסל עדכון של סניף אחד יכול לקחת כמה דקות. **עם שרת**, השרת כבר הוריד את הקבצים בבוקר, והטלפון מוריד אותם ממנו תוך שניות (בבדיקה: 18 שניות לסניף שופרסל). אם השרת לא מכיר סניף מסוים או לא זמין, האפליקציה חוזרת לבד להורדה ישירה מהרשת.

3. האפליקציה: מה קורה בפנים

בחירת סניפים

בפתיחה הראשונה האפליקציה מורידה את קובצי ה-Stores של כל 19 הרשתות, כ-1,585 סניפים. כל רשת נטענת בנפרד ובמקביל, כך שהרשתות המהירות מופיעות תוך שניות גם כששופרסל עדיין נטענת. יוחננוף, פוליצר, סאלח דבאח ורוב קשת טעמים לא מציינות עיר, ולכן הסניפים שלהן מופיעים בלי שם עיר. סמל היישוב מתורגם לשם עיר מתוך טבלת היישובים של הלמ"ס שמצורפת לאפליקציה (1,316 יישובים). בוחרים עד 12 סניפים. אפשר גם למיין לפי מרחק ("חנויות קרובות אליי"): האפליקציה מקבלת מיקום מהטלפון, מאתרת כל עיר פעם אחת בשירות איתור הכתובות של הטלפון, ואז מאתרת את כתובות הסניפים בערים הקרובות. כל התוצאות נשמרות, כך שהחיפוש הבא מהיר.

עדכון המחירים, שלב אחר שלב

העדכון רץ ברקע (WorkManager) פעם ביום, מיד אחרי שבוחרים סניף חדש, או בלחיצה על כפתור הרענון:

1. **איתור הקובץ החדש:** לכל סניף שנבחר, האפליקציה מבקשת את רשימת הקבצים (מהשרת או מאתר הרשת; באתר המשותף קודם מתחברים) ומוצאת את קובץ ה-PriceFull וה-PromoFull העדכניים לפי התאריך שבשם הקובץ.
2. **דילוג על מה שכבר יש:** אם התאריך זהה לקובץ שכבר נטען, הסניף מדולג. כך עדכון שני באותו יום כמעט לא מוריד כלום.
3. **הורדה ופענוח בזרימה:** הקובץ נקרא תוך כדי הורדה, בלי לשמור אותו: פריסת gzip או zip, זיהוי הקידוד (UTF-8, או UTF-16 כמו ברשימת הסניפים של קרפור), וקריאת הרשומות אחת אחת. כך גם קובץ של 15,000 מוצרים לא תופס זיכרון רב.
4. **החלפה במאגר:** המחירים הישנים של הסניף מוחלפים בחדשים בפעולה אחת (טרנזקציה). אם ההורדה נקטעת באמצע, נשארים המחירים הקודמים.
5. **רישום היסטוריה:** לפני ההחלפה האפליקציה משווה כל מחיר חדש לישן. כשמחיר השתנה, נשמרים המחיר הקודם והחדש בטבלת ההיסטוריה, וכך נבנה גרף המחירים בדף המוצר.
6. **מבצעים:** אותו תהליך לקובץ המבצעים: כל מבצע נשמר עם תאריכיו ורשימת הברקודים שבו.
7. **בדיקת מעקב:** בסוף העדכון נבדק כל מוצר במעקב. אם המחיר הזול ביותר ירד, או הגיע למחיר היעד, נשלחת התראה, ולחיצה עליה פותחת את דף המוצר.

עד שלושה סניפים מתעדכנים במקביל, כי רוב הזמן הולך על המתנה לאתר הרשת ולא על ההורדה עצמה.

המאגר בטלפון

טבלה	מה נשמר
stores / my_stores	כל הסניפים, והסניפים שבחרתם עם תאריך הקובץ האחרון שנטען
items	המחיר הנוכחי של כל מוצר בכל סניף שנבחר (ברקוד, שם, יצרן, מחיר, מחיר ליחידה)
history	שינויי מחיר לאורך זמן, לכל מוצר וסניף
promos / promo_items	המבצעים, והברקודים שכל מבצע חל עליהם
shopping_list / watch	רשימת הקניות שלכם, והמוצרים במעקב עם מחיר היעד והמחיר האחרון שנראה
geo	מיקומי ערים וסניפים שכבר אותרו

המסכים

- **חיפוש:** מחפש בכל המוצרים של הסניפים שלכם. ספרות בלבד נחשבות ברקוד (גם בלי אפסים מובילים); אחרת כל מילה צריכה להופיע בשם או ביצרן. התוצאות מקובצות לפי ברקוד, עם המחיר הזול ומספר הסניפים.
- **סורק:** המצלמה מזהה ברקוד (EAN-13, EAN-8, UPC, Code 128) בעזרת מודל ML Kit שמובנה באפליקציה, ופותחת את המוצר או מוסיפה אותו לרשימה.
- **דף מוצר:** מחיר בכל סניף מהזול ליקר, מחיר ליחידת מידה, מבצעים פעילים, וגרף היסטוריה עם קו לכל סניף.
- **מבצעים:** מבצעים שעוד לא הסתיימו, עם חיפוש וסינון לפי סניף, ורשימת המוצרים בכל מבצע.
- **מעקב:** המוצרים במעקב, המחיר הנוכחי והיעד.

איך מחושבת רשימת הקניות

לכל סניף מחושב סכום של מחיר × כמות לכל מוצר שנמכר בו. הסניפים מדורגים קודם לפי מספר המוצרים החסרים, ורק אחר כך לפי הסכום, כדי שסניף שנראה זול רק כי חסרים בו חצי מהמוצרים לא יופיע ראשון. בנוסף מחושבת "קנייה מפוצלת": כל מוצר בסניף שבו הוא הכי זול.

מוצר (כמות)	סניף א	סניף ב	סניף ג
חלב (2)	₪6.00	₪5.00	**₪1.00**
לחם (1)	**₪10.00**	₪12.00	חסר
מוצר נדיר (1)	חסר	**₪30.00**	חסר
סכום ודירוג	₪22 · חסר 1 · מקום 2	₪52 · הכול · **מקום 1**	₪2 · חסרים 2 · מקום 3

בדוגמה, הקנייה המפוצלת (המודגשים) עולה ₪42 בשלושה סניפים. הסכומים לפי המחיר הרגיל; מוצר שיש עליו מבצע מסומן ב-★.

4. השרת: מה קורה בפנים

השרת כתוב ב-Python ומשתמש רק בספרייה הסטנדרטית (בלי התקנות). הוא רץ כתהליך אחד עם שלושה חלקים: אוסף שרץ לפי לוח זמנים, מאגר SQLite, ושרת HTTP שעונה לבקשות. ההגדרות קובעות אילו סניפים לשמור: כולם, רשת שלמה, או רשימת סניפים.

האיסוף היומי, שלב אחר שלב

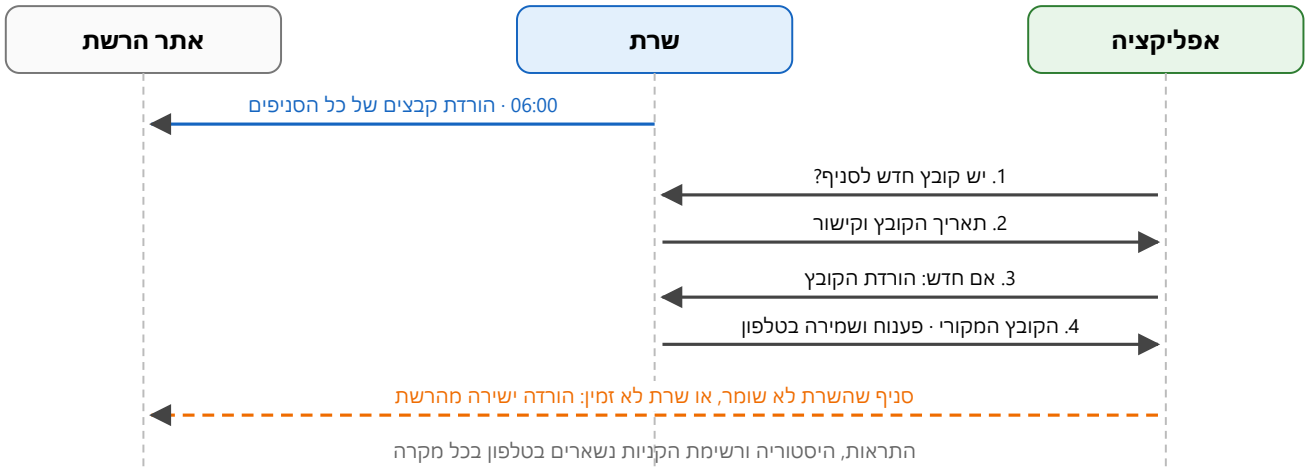
1. **תזמון:** איסוף אחד מיד כשהשרת עולה, ואחר כך כל יום ב-06:00 שעון ישראל, אחרי שהרשתות מפרסמות את הקבצים המלאים (בערך 04:00–05:00). אף פעם לא רצים שני איסופים במקביל.
2. **רשימת הסניפים:** הורדת קובץ ה-Stores של כל רשת ועדכון טבלת הסניפים.
3. **רשימת הקבצים:** כשנשמרים כל הסניפים של רשת, השרת קורא את רשימת הקבצים המלאה שלה (בשופרסל 22 עמודים לכל סוג קובץ; באתר המשותף ובאתר binaprojects רשימה אחת) ומוצא את הקובץ העדכני של כל סניף. כשנבחרו רק כמה סניפים, הוא שואל על כל סניף בנפרד, וזה מהיר יותר.
4. **הורדה מקבילית:** ארבע הורדות בו-זמנית. קובץ שכבר יש לו את אותו תאריך מדולג. בכשל יש עד שלושה ניסיונות עם המתנה הולכת וגדלה.
5. **שמירה כפולה:** הקובץ המקורי נשמר בתיקיית המראה כפי שהרשת פרסמה אותו (זה מה שהאפליקציה מורידה), והתוכן מפוענח ונטען למאגר, כולל רישום שינויי מחיר להיסטוריה.
6. **סיכום:** מספר הסניפים שעודכנו, אילו נכשלו, ומשך האיסוף נשמרים ומוצגים ב-`/api/status` ובראש דף האינטרנט.

מה השרת מגיש

כתובת	מה מקבלים
`GET /api/files/{chain}/{store}`	פרטי הקבצים השמורים של סניף (תאריך, גודל וקישור). כך האפליקציה יודעת אם יש קובץ חדש.
`GET /files/{chain}/{store}/{kind}`	הקובץ עצמו, בדיוק כפי שהרשת פרסמה אותו, כך שהאפליקציה מפענחת אותו באותו קוד
`GET /api/search?q=…`	חיפוש מוצרים לפי שם או ברקוד בכל הסניפים, עם המחיר הזול והיקר
`GET /api/products/{barcode}`	מחיר המוצר בכל סניף, מבצעים פעילים והיסטוריית מחירים
`POST /api/basket`	חישוב סל קניות, באותו אלגוריתם כמו באפליקציה
`GET /api/promos` · `/api/stores` · `/api/chains`	מבצעים פעילים, רשימת סניפים, ורשתות
`GET /api/status`	מצב השרת: כמה סניפים ומחירים יש, מתי עודכן, והאם איסוף רץ עכשיו
`POST /api/sync`	הפעלת איסוף ידנית. מותרת רק מהמחשב עצמו, או עם סיסמת מנהל
`GET /`	דף אינטרנט בעברית לחיפוש מוצרים והשוואת מחירים

תשובות JSON גדולות נשלחות דחוסות (gzip), והשרת עונה לכמה בקשות במקביל.

5. סנכרון האפליקציה דרך השרת



תרשים 2: עדכון סניף כשמוגדר שרת. השרת מוריד מהרשת פעם ביום; הטלפון מוריד מהשרת.

הגדרת השרת נעשית בלשונית "חנויות" ← "מקור המחירים", בכתובת כמו `http://192.168.1.23:8080` (המחשב והטלפון באותה רשת Wi-Fi). בשמירה האפליקציה בודקת שהשרת עונה ומציגה כמה סניפים יש בו.

6. אמינות ומגבלות

- **אתר שופרסל:** המתנה של עד 150 שניות לתשובה ושלושה ניסיונות. ללא שרת, עדכון סניף שופרסל יכול לקחת כמה דקות.
- **רשתות שעוד חסרות.** ויקטורי, ח. כהן ומחסני השוק מפרסמות באתר (laibcatalog) שנותן קבצים רק דרך טופס רב-שלבי. האתרים של סופר יודה ומשנת יוסף לא ענו. קופיקס, גוד פארם, יילו ועוד כמה כבר לא נכנסות עם שמות המשתמש הישנים שלהן באתר המשותף. כל רשת נוספת היא רכיב מקור אחד בכל צד.
- **תעודת אבטחה.** האתר המשותף משתמש בתעודת שורש (SSL.com TLS RSA Root CA 2022) שאין באנדרואיד 14. האפליקציה כוללת את התעודה הזו, שנלקחה ממאגר התעודות המהימנות של Windows, וסומכת עליה רק עבור האתר הזה. כל החיבורים עדיין מאומתים במלואם.
- **מבצעים לא נכנסים לסכום הסל.** כללי המבצעים מגוונים (כמויות, מועדונים, קופונים), ולכן הם מוצגים ומסומנים אבל לא מחושבים.
- **היסטוריה מתחילה ביום ההתקנה.** הגרף מתמלא ככל שהמחירים משתנים בעדכונים היומיים.
- **השרת מקומי.** הטלפון משתמש בו רק כשהמחשב פועל ונגיש. כשהוא לא זמין, האפליקציה מורידה ישירות מהרשתות.